基于vibecoding项目的全面存档——老人防摔智能识别

摘要

跌倒是老年人居家安全的重大威胁。本项目以"Vibe Coding"（AI 协作编程）方式，借助 AI 编程助手完成了一个老人跌倒检测系统的完整开发：从环境搭建、模型验证、规则式检测、多轮调试排错，到自定义数据集微调训练自己的检测模型，再到基于人体关键点运动学的跌倒确认算法，最后接入真实摄像头迭代出"地板语义分割 + 三维姿势估计"的多判据融合方案。项目形成了三条可对比的技术路线：①姿势关键点+规则判断；②端到端自训练检测模型（YOLO26n 微调，fall 类 mAP@50 达 0.97）；③几何+运动学双重判据确认。在 UR Fall 公开数据集上的对比实验表明：三维姿势引擎（MediaPipe，33 关键点含深度）对 2D 盲区（朝镜头方向摔倒、俯拍机位）优势显著（检出率 7/8 vs 4/8），而"躯干贴地 + 运动学速率"的多判据融合实现了日常活动零误报。系统最终落地为完整闭环：摔倒确认后通过推送管道向自研 Android App 发送强提醒（响铃、全屏警报、现场截图、自动拨打紧急联系人），并支持 App 内实时视频监控，端到端延迟约 2~3 秒。本项目同时是一次 Vibe Coding 方法论的真实记录：AI 负责代码实现与调试，人负责需求定义、效果验收与方向决策。

关键词：跌倒检测；YOLO；姿势估计；迁移学习；Vibe Coding；人机协作

---

1. 引言

1.1 项目背景

跌倒是 65 岁以上老年人意外伤害的首要原因之一。独居老人跌倒后若无法及时起身和求助，可能延误救治。基于计算机视觉的非接触式跌倒检测，无需穿戴设备、保护隐私、成本低廉，是智慧养老的重要研究方向。

1.2 Vibe Coding 课程要求

本项目全程采用 Vibe Coding 模式开发：开发者不直接编写大部分代码，而是通过自然语言向 AI 编程助手描述需求、验收运行结果、反馈问题现象，由 AI 完成代码编写、环境配置、错误诊断与迭代修复。本文既报告跌倒检测系统本身的技术实现，也记录这一新型开发模式下的完整协作过程与经验。

1.3 开发环境

- 硬件：AMD Radeon RX 7700 XT（12GB）、Intel i5-12600KF、64GB 内存
- 软件：Windows 11、Python 3.11、Ultralytics 8.4.152、PyTorch 2.14（CPU 版）
- 模型：YOLO26n-pose（官方姿势估计）、YOLO26n（自训练检测模型）、MediaPipe Pose Landmarker（三维姿势）、SegFormer-b0（地板语义分割）

---

2. 技术路线总体设计

项目最终形成三条可对比的技术路线（整体流程见图 1）：

路线	原理	优点	缺点
①姿势+规则	关键点→宽高比/躯干角/下坠速度规则	可解释、无需训练	阈值依赖场景，鲁棒性一般
②自训练检测模型	YOLO26n 在跌倒数据集上微调，直接输出 fall 类	准确率高、逻辑简单	黑盒、需要标注数据
③运动学确认	姿态几何 + 躯干贴地 + 角速度/下落速度（3D 版为竖直性速率）多重判据	可解释、能区分"摔倒/自行躺下/弯腰"	对关键点抖动敏感

系统整体流程：视频帧 → 人体检测/姿势估计 → 特征提取 → 状态机判定 → 声光报警。

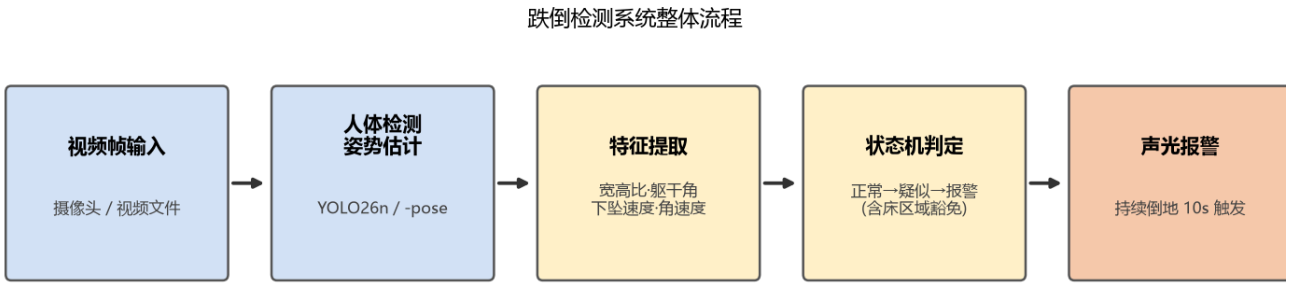


图 1　跌倒检测系统整体流程

2.1 YOLO26n 检测模型的内部结构

本项目使用的两个模型（检测模型 YOLO26n、姿势模型 YOLO26n-pose）都属于 YOLO 系列**单阶段检测器**：整张图片只完整过一遍神经网络，就一次性输出所有目标——这也是 "You Only Look Once" 名字的由来。与"先提候选区域、再逐个分类"的两阶段方法相比，单阶段设计是其实时性的来源。网络整体可分为三段（图 2）：

- **Backbone（主干网络，特征提取）**：由卷积层逐级下采样，把 640×640 的图片压缩成越来越小的特征图（160→80→40→20），通道数同步增加（最多 256）。直观地说，浅层卷积学会辨认边缘、颜色、纹理，深层卷积学会辨认"人形"这类整体结构。核心构件是 **C3k2 模块**（把特征分两路、一路走瓶颈卷积一路直通，再合并——既省算力又保证梯度流通）；最深处还有 SPPF（空间金字塔快速池化，用不同尺度的池化扩大"视野"）和 **C2PSA 注意力模块**（让网络学会"盯"画面中的关键区域而不是平均用力）。
- **Neck（颈部，多尺度融合）**：采用 PAN-FPN 结构——先自顶向下上采样（把深层的"语义理解"带回大分辨率特征图），再自底向上下采样（把浅层的"细节纹理"补回去），深浅层特征逐层拼接。最终产出三个尺度的融合特征：**P3（80×80，负责小目标）**、**P4（40×40，中目标）**、**P5（20×20，大目标）**。人远处小、近处大都能被检出，靠的就是这种多尺度设计。
- **Head（检测头，解耦输出）**：三个尺度各配一组**解耦头**——"回归分支"和"分类分支"分开预测：前者用两层卷积回归检测框坐标（x，y，w，h），后者用深度可分离卷积（DWConv）输出每个类别的置信度。YOLO26 还采用一对一标签分配实现**端到端输出，无需 NMS 去重**这一传统后处理。

整个模型仅 257 万参数（5.4 MB），是这个量级里典型的"小而全"设计。记住这个三段结构——3.5 节讲微调时，"预训练保留了什么、新训练改了什么"就是按这三段来说的。

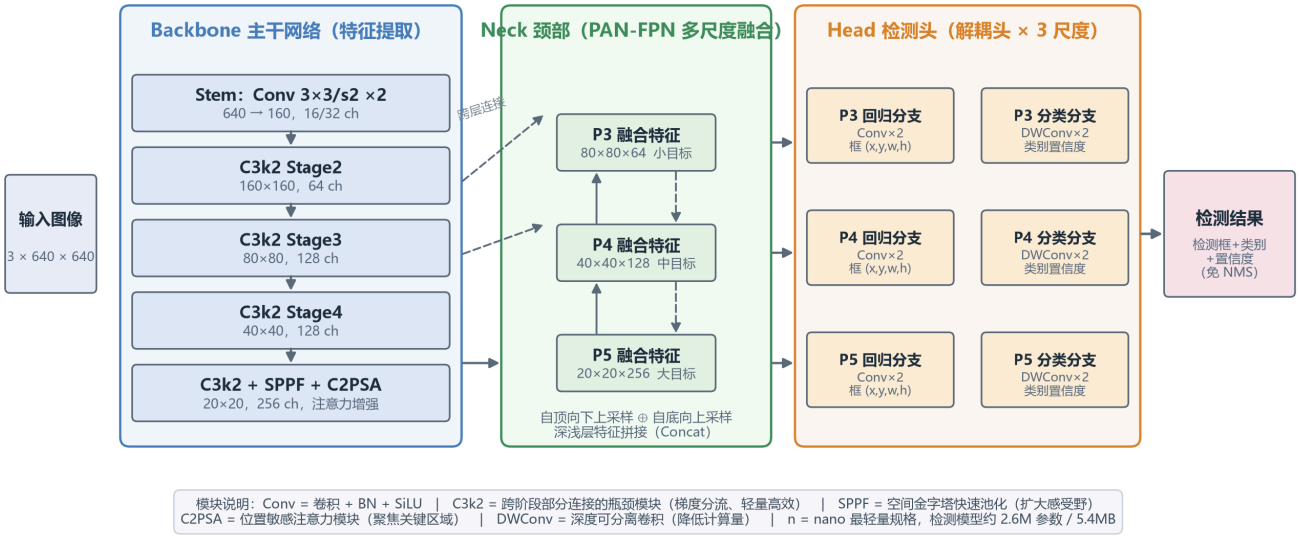


图 2　YOLO26n 网络结构：Backbone（特征提取）→ Neck（PAN-FPN 多尺度融合）→ Head（三尺度解耦检测头）。图中模块名称与通道数取自本项目所用权重的实际结构

2.2 姿势估计模型如何"看到"关节

YOLO26n-pose 回答的是一个更细的问题：不止"人在哪"，还有"这个人的每个关节在哪"。它采用的是 Top-Down（自顶向下）路线，分两步走（图 3）：

1. **先检测人**：与检测模型共享同一套 Backbone + Neck，由检测分支输出每个人的人体框和置信度；
2. **再回归关键点**：对每个检测到的人，**关键点分支**（cv4）直接回归出 17 个关节点的坐标，每个点输出三元组 `(x, y, σ)`——坐标加该点自身的置信度（被遮挡的点 σ 会明显变低，下游可据此过滤）。

两点值得说明：其一，关键点是**按检测框独立回归**的，17 个点一次算完，不存在"在图里找点再拼人"的搜索过程，因此速度快、且天然支持多人（多一个框就多一组 17 点）；其二，姿势

模型与检测模型是"同一个身体、换了个头"的关系——主干和颈部完全共享，只多了关键点分支，参数量从 257 万增至 368 万（7.5 MB）。

需要强调的是，**模型的能力到此为止：它只负责报出 17 个坐标，并不理解这个人在干什么**。画面上明明有人摔倒，它也只是默默画骨架，不会报警。"这些点意味着什么"要靠我们自己定义——这正是 3.6 节运动学分析层要做的事。

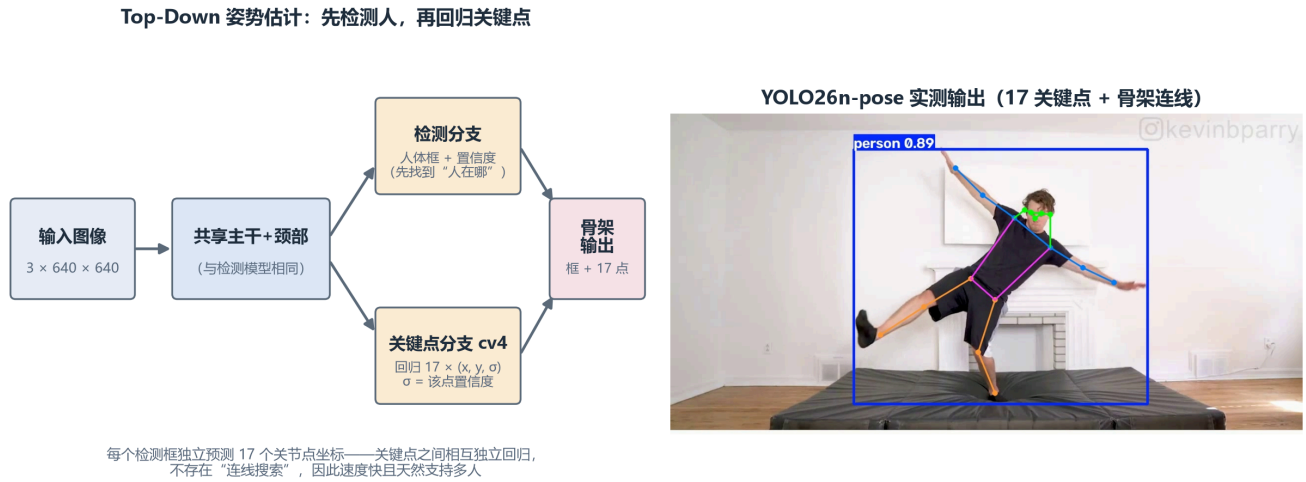


图 3　Top-Down 姿势估计原理（左）与 YOLO26n-pose 在测试视频上的实测输出（右）：先检测人体框，再对每个框回归 17 个 COCO 关键点及其置信度

---

3. 实现过程（Vibe Coding 迭代记录）

3.1 第一轮：环境搭建与模型验证

提出需求"yolo 可以识别人体姿势吗"后，AI 助手在本地创建 Python 虚拟环境、安装 Ultralytics，并对用户已有的 `yolo26n-pose.pt` 做了加载与推理验证：在测试图上成功检测 5 个人、每人 17 个 COCO 关键点，CPU 推理约 67ms/帧，满足准实时要求。

Vibe Coding 要点：AI 直接完成环境配置与验证，人只需确认"能不能用"。

3.2 第二轮：规则式跌倒检测原型

实现了基于宽高比与躯干倾斜角的跌倒判断脚本，支持摄像头/视频两种输入。第一版很快能跑，但测试暴露出问题：**"摔倒"和"睡觉"在单帧画面上都是水平姿态，无法区分**。

3.3 第三轮：引入时间与空间上下文

针对"摔倒 vs 睡觉"问题，系统升级为：

- **时间维度**：人物追踪 + 重心下坠速度检测（摔倒=突然下坠，睡觉=缓慢躺下）
- **空间维度**：床区域 ROI 标定（鼠标点两下标记床的位置，躺床上永不报警）
- **状态机**：正常 → 疑似 → 报警，倒地后持续不动 N 秒才正式报警

3.4 第四轮：真实视频调试中的三个关键 bug

用网络下载的摔倒视频实测时发现"摔倒了却不报警"。AI 助手编写诊断脚本输出逐帧测量数据（下坠速度、宽高比、躯干角），定位出三个问题：

1. **时间基准 bug（最隐蔽）**：视频文件处理时混用了墙上时钟与视频时间戳。CPU 处理速度慢于视频帧率，导致测出的下坠速度被"摊薄"为真实值的一半左右，永远达不到阈值。修复：视频文件一律使用视频内时间戳。
2. **检测闪烁导致轨迹历史被清空**：人摔倒瞬间姿态剧变，检测器容易丢帧；原代码一帧丢失即删除该人全部历史，下坠速度无法累计。修复：轨迹保留 2 秒宽限期。
3. **追踪 ID 频繁切换**：躺倒的人被反复分配新 ID，倒地计时器随之清零。修复：ID 合并机制（新 ID 出现在旧轨迹 200px 内则继承状态）。

为进一步摆脱对追踪 ID 的依赖，最终引入**区域倒地计时**：不再关心"是谁"，只关心"某个位置是否持续有人躺倒超过 10 秒"，ID 怎么换都不影响计时。

Vibe Coding 要点：人负责报告现象（"摔倒了不识别"），AI 负责写诊断工具、读数据、定位根因。调试效率取决于人描述现象的准确程度。

3.5 第五轮：自训练检测模型（迁移学习/微调）

按课程"模型训练"要求，本轮目标是**训练一个属于自己的跌倒检测模型**。考虑到部分同学没有深度学习基础，本节先把"微调"这件事讲清楚，再介绍具体做法。

3.5.1 什么是"微调"（Fine-tuning）？

直接从零训练一个检测模型，需要几十万张标注图片和昂贵的 GPU 集群，个人电脑完全无法承受。但实际上没人这么做——大家的做法是**站在巨人肩膀上**：

- Ultralytics 官方已经在 **COCO 数据集**（约 12 万张标注图片、80 个类别：人、车、猫、狗、杯子……）上训练好了通用模型，并把训练成果打包成权重文件 `yolo26n.pt` 公开下载。这就叫**预训练权重**（pretrained weights）。
- 这个模型已经学会了"看图"的基本功：哪里有物体、物体的轮廓和纹理长什么样。它欠缺的只是：**不认识"摔倒"这个类别**。
- 于是我们在它的基础上，用几百张跌倒场景的图片**继续训练一小段时间**，让它把已有的"看图能力"迁移到我们的新任务上。这就叫**迁移学习**（Transfer Learning），工程上的具体做法就是**微调**（Fine-tuning）。

一个直观的类比：预训练模型像一位见多识广、基本功扎实的美术生，微调就是给他上一门 48 分钟的"跌倒场景速成专业课"——不用重学素描，只要学会辨认新题材。

结合 2.1 节的三段结构，微调时网络里实际发生的事可以说得很具体（图 4）：**Backbone 和 Neck 的全部权重直接继承**——边缘、纹理、人体轮廓这些通用特征 374 张小数据集根本学不出来，必须复用；**唯一需要重建的是 Head 分类分支的最后一层卷积**，因为类别数从 80 变成了 3，输出通道数对不上，这一层从头初始化；训练时全部层以小学习率（0.01 起步，线性衰减至 6.1×10^{-5}）继续更新，让已有特征温和地向跌倒场景适配，而不是推倒重来。

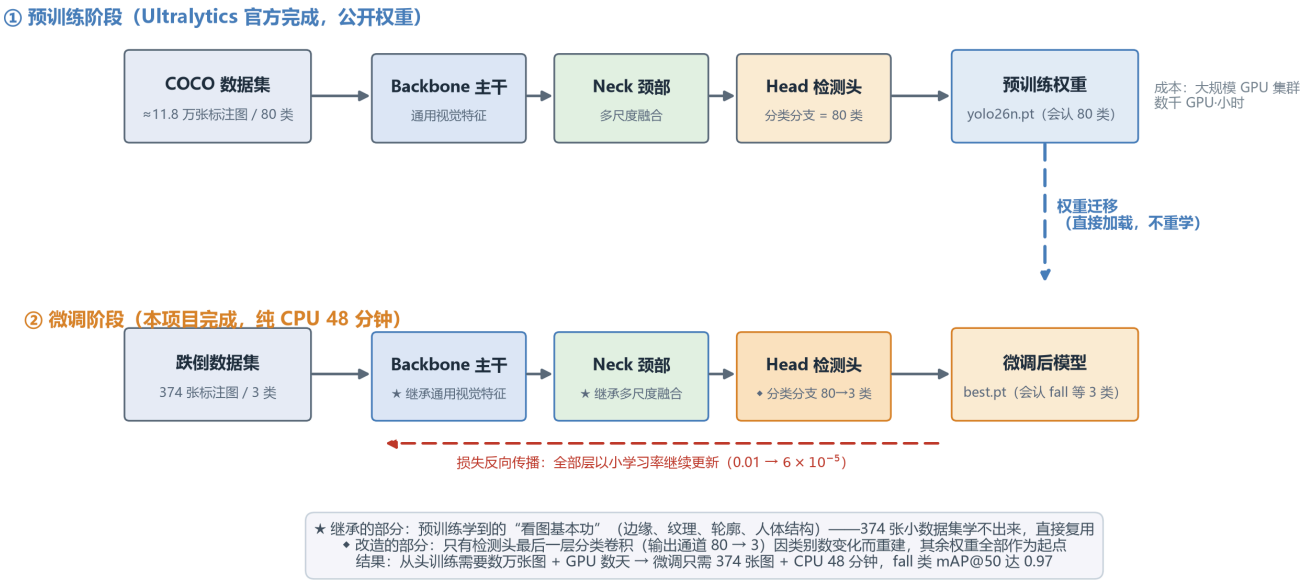


图 4　迁移学习（微调）原理：Backbone/Neck 的通用特征被完整继承，仅检测头分类分支因类别数变化（80→3）而重建

3.5.2 数据集准备与核验

数据集从 HuggingFace 镜像站获取（公开跌倒检测数据集，374 张训练图 / 111 张验证图，YOLO 格式，3 个类别：fall 躺倒 / standing 站立 / sitting 坐着）。每张图片配一个同名 `.txt` 标注文件，每行记录一个目标：`类别编号 中心x 中心y 宽 高`（均为 0~1 的归一化坐标）。

训练前做了两项核验，防止"垃圾进、垃圾出"：

- **标注格式核验**：随机抽取图片，把标注框按类别着色后叠加显示（红=躺倒、绿=站立、蓝=坐着），肉眼确认框的位置和类别语义都对得上，避免"类别标错"这种低级但致命的事故；
- **类别合法性检查**：脚本扫描全部标签文件，统计类别索引分布，确认没有超出 0~2 范围的非法编号、没有空标注。

3.5.3 训练配置

配置项	取值	说明
基础权重	`yolo26n.pt`（官方预训练）	微调的起点，见 3.5.1
epochs	30	整个训练集完整"过"30 遍
imgsz	640	输入图片统一缩放到 640×640
batch	8	每次迭代喂 8 张图（CPU 内存所限）
初始学习率 lr0	0.01	见 4.1.1 学习率曲线分析
优化器	auto（SGD，动量 0.937）	Ultralytics 自动选择
数据增强	mosaic、水平翻转、HSV 色彩抖动、随机缩放平移	用" artificially 变形"的图片扩充数据多样性，防止死记硬背
硬件	纯 CPU（i5-12600KF）	全程约 48 分钟

一句话总结训练过程：`model.train(data="fall.yaml", epochs=30, imgsz=640, batch=8)`——在 Vibe Coding 模式下，我需要做的只是向 AI 描述"用这个数据集微调 yolo26n，30 轮"，AI 负责写好脚本、处理路径与依赖，我盯着训练日志确认曲线正常即可。训练过程中的曲线分析见 4.1 节。

3.6 第六轮：运动学跌倒确认

3.6.1 先搞清楚：什么是"关键点"？

前面用到的 `yolo26n-pose.pt` 是一个**姿势估计（Pose Estimation）模型**（识别原理见 2.2 节）。它检测到人之后，不只是画一个框，还会在人身上标出 **17 个预设的身体关节点**（COCO 标准）：鼻子、双眼、双耳、双肩、双肘、双腕、双髋、双膝、双踝。每个关键点就是一组像素坐标 `(x, y)`（另带该点置信度），模型每帧都输出这 17 个点的位置。

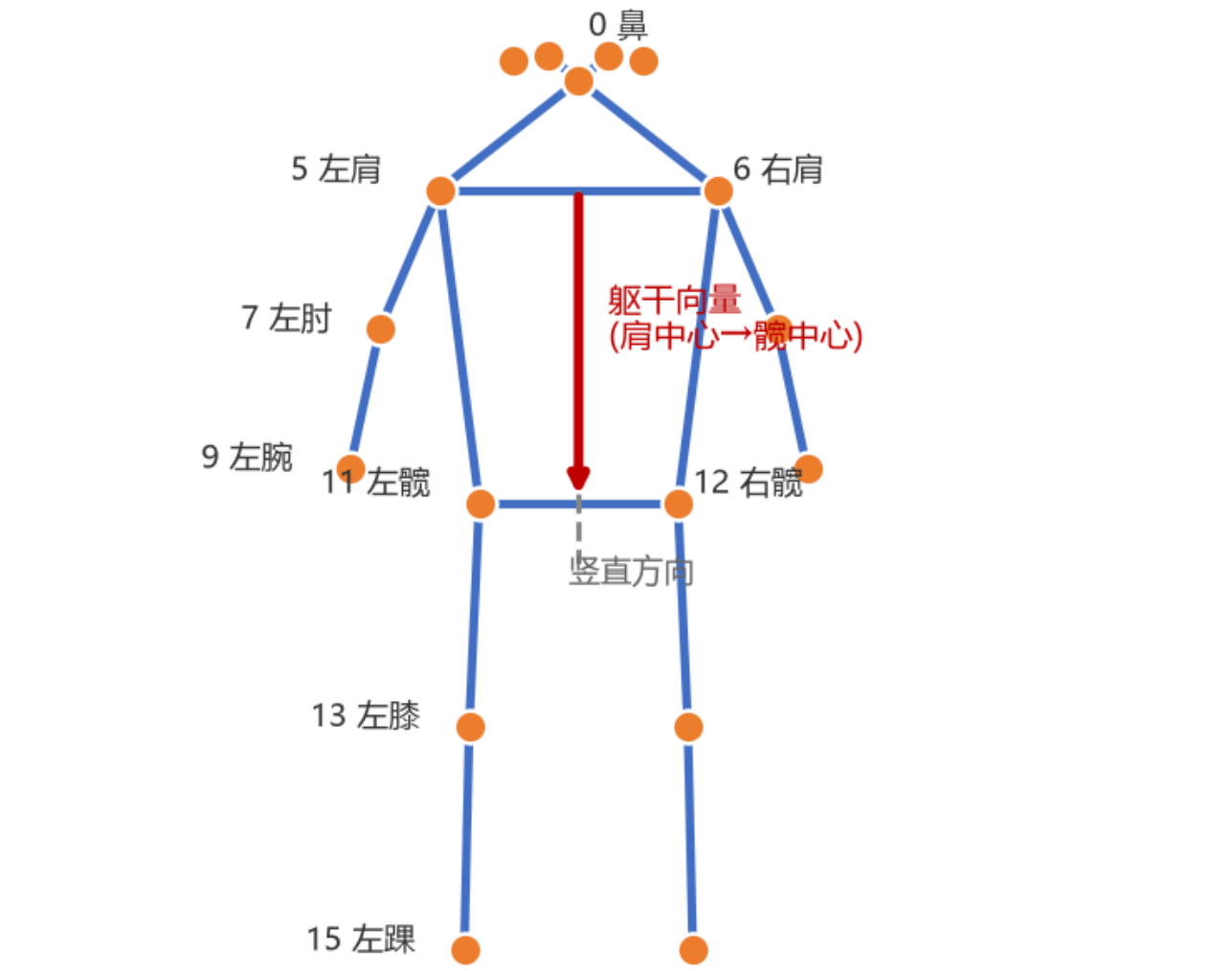


图 5　COCO 17 人体关键点与本文使用的"躯干向量"

换句话说，姿势模型把"一个人"从一整张复杂图片，压缩成了 17 个坐标点——**这就是机器能看懂的人体表示**。但它的能力到此为止：它只负责报点，**不会告诉我们这个人在干什么**。画面上明明有人摔倒了，它也只是默默画骨架，不会报警。

3.6.2 在关键点之上加了什么：动作分析层

本轮新增的是一个**纯几何/运动学计算层**——不新增任何神经网络，而是拿关键点坐标做数学计算，自己定义"什么叫摔倒"：

- **姿态分类（看终态）**：取左右肩的中点、左右髋的中点，连成一条"躯干向量"（图 5 中红色箭头），计算它与竖直方向的夹角；再结合检测框宽高比，实时判定 站立 / 弯腰 / 坐着 / 躺倒。例如躯干角接近 90° 且检测框"横宽竖窄"，就是躺倒；
- **跌倒确认（看过程）**：当某人刚变为"躺倒"姿态时，回溯前 2.5 秒内逐帧记录的关键点坐标，计算
- 峰值重心下落速度 v（画面高度/秒）——髋部中点向下移动的速率
- 峰值躯干角速度 ω（度/秒）——躯干向量转动的速率
- 判据：v > 0.30 或 ω > 60°/s ⇒ 确认为摔倒；否则判定为"自行缓慢躺下"，不报警。

这一"几何终态 + 运动学过程"的双重判据，从物理上区分了摔倒与睡觉/躺下：睡觉过程缓慢，两项运动学指标均不达标，天然被过滤。

3.7 第七轮：真实摄像头实测与"弯腰误报"问题

接入 USB 摄像头（高角度斜俯机位，覆盖房间地面）实测时，暴露出一个视频测试中不明显的问题：**弯腰捡东西会被误判为摔倒**。分析原因：弯腰时躯干角同样接近 90°、角速度也不小，单看这两项指标与摔倒高度重叠。两轮修正：

1. **躺倒判定收紧**：从"躯干角 > 55° **或** 检测框横宽"改为"躯干角 > 55° **且** 框横宽"——弯腰时躯干虽弯，人仍是"竖着一坨"，检测框不会变宽；
2. **持续确认**：躺倒姿态需持续 0.5 秒才触发分析——弯腰是瞬时动作，摔倒是持续状态。

误报有所缓解，但弯腰到最低点时（手接近地面）仍可能触发。这促使我们思考更本质的判据。

3.8 第八轮：地板语义分割——让姿势与地面产生联系

摔倒的终态定义只有一个：**躯干接触地面**。而弯腰时无论手离地面多近，**肩、髋、头始终悬在地板上方**。要让系统利用这一区别，必须先让"地面"在画面中有明确的数学定义。单目 RGB 相机没有深度信息，我们采用**语义分割**方案：

- **地板分割模型**：SegFormer-b0（ADE20K 数据集训练，150 类中自带 floor 类，无需自行训练），对画面分割出地板区域；固定摄像头下地板静止，启动时分割一次、后台线程每 60 秒（或画面无人时）刷新即可，不阻塞检测主线程；
- **距离场**：对地板掩码做距离变换，任意像素可查询"到最近地板像素的距离"；
- **身高标尺**：系统在人正常站立时自动记录"脚踝→髋部"像素距离（站立时脚踝必在地面上，这是零成本的自校准）；
- **躯干三点贴地校验**：疑似躺倒时，分别计算髋（< 0.30 个腿长）、肩（< 0.45）、鼻/头（< 0.55）到地板的距离——**三点全部贴地才进入运动学确认**；弯腰时手可以摸地，但躯干三点悬空，直接排除。关键点触地状态在画面上实时可视化（绿点=触地，灰圈=悬空）。

3.9 第九轮：三维姿势估计（MediaPipe 33 关键点）

实测与复盘又揭示了 2D 姿势估计的一个结构性盲区：**人朝/背摄像头方向摔倒时，画面投影中躯干角几乎不变**（UR Fall fall-03 片段中全程仅约 15°），2D 判据必然失效。为此引入 MediaPipe Pose Landmarker：

- 输出 **33 个三维关键点**（在图像 x/y 之外，每个点带 z 深度分量，表示朝向/远离摄像头）；
- 核心指标改为**躯干竖直性**：verticality = |Δy| / |三维躯干向量|——站立时 ≈ 0.9，任何方向躺倒时（包括朝镜头方向）≈ 0.1；
- 判定链与 2D 版完全一致（姿态持续 0.5 秒 → 躯干三点贴地 → 运动学确认），仅运动学指标把"躯干角速度"换成"竖直性下降速率"（阈值 0.8/秒）。

3.10 第十轮：远程报警——推送管道、自研 App 与监控画面

检测只是前半场，**报警必须送达到人**。本轮把系统从"本地演示"推进到"远程落地"，最终形态：摔倒确认的瞬间，家属手机响铃震动、全屏红色警报、附现场截图、可一键查看实时监控画面、并可自动拨打紧急联系人电话。

报警通道的选型过程本身就是一次工程权衡：语音呼叫中心方案（腾讯云 CCC、阿里云 VMS）面向企业客户且需话术审核备案，个人开发者门槛高；最终采用分层架构——

1. **推送管道（ntfy.sh 开放协议）**：检测端确认摔倒后，向 ntfy 公共推送服务器的一个私密主题（长随机串即凭据）POST 消息，紧急优先级 + 现场截图附件合并为单条消息。该通道走互联网，4G 外网同样可达；
2. **自研 Android App（Kotlin，零第三方依赖）**：前台服务通过 SSE 长连接订阅报警主题；收到消息即循环响铃 + 震动 + 全屏红色警报页（锁屏也可弹出），警报页内嵌现场截图、"查看实时监控"入口和"呼叫家属"按钮（5 秒后自动拨打预设紧急联系人，走本机 SIM 卡通话，零费用）；
3. **实时监控画面（MJPEG 视频流）**：检测端内置轻量 HTTP 服务器，把叠加了骨架与判定信息的画面以约 12 fps 推流，App 内"监控画面"页逐帧解码播放，效果等同家用摄像头 App。

开发中踩过的坑（Vibe Coding 记录）：安卓 9+ 默认禁止明文 HTTP 导致监控画面黑屏（`Cleartext HTTP not permitted`），加 `usesCleartextTraffic` 配置解决；24 小时保活问题曾考虑"定时看门狗 + 划掉重生"方案，但这类"流氓式保活"违背正规应用设计，最终只保留官方机制（前台服务 + START_STICKY + 开机自启 + 用户授权电池白名单），并预留 4G 电话模块（SIM800C，AT 指令拨号 + 中文 TTS 播报，代码已就绪）作为完全不依赖手机在线状态的硬件兜底通道。

3.11 端到端验收

用 UR Fall 数据集的摔倒视频驱动完整系统（不依赖真人摔倒），连续两次全链路通过：播放 fall-12 / fall-08 → 竖直性从 0.71 跌至 0.02 → 持续确认 + 运动学复核通过 → 手机响铃弹窗（含现场截图）→ 监控画面可实时查看。从画面中人倒地到手机收到警报，端到端延迟约 2~3 秒。

---

4. 实验与结果

4.1 训练过程分析（学习率与损失曲线）

训练共 30 轮（epoch），纯 CPU 耗时 2892 秒（约 48 分钟）。Ultralytics 每轮自动记录学习率、各项损失与验证集精度，完整曲线如图 6 所示（蓝色为每轮实测值，橙色虚线为平滑

趋势）。

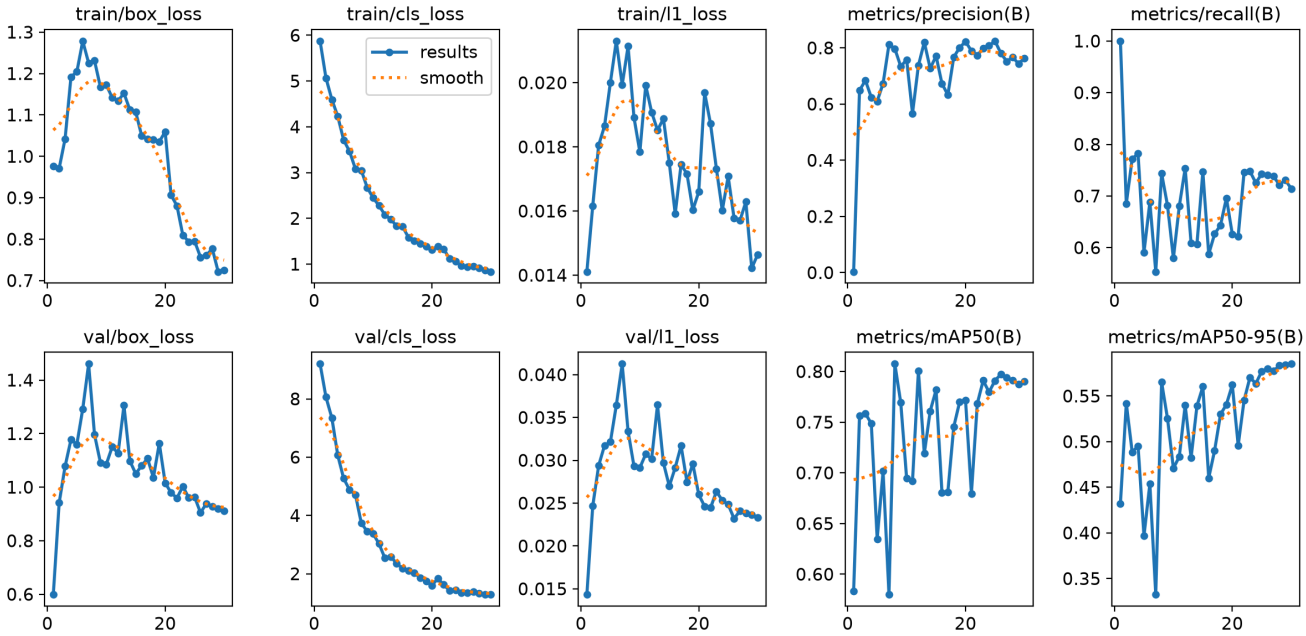


图 6　训练过程曲线：上排为训练集指标，下排为验证集指标

4.1.1 学习率曲线：先热身，再衰减

学习率（learning rate）可以理解成"模型每一步改正错误的幅度"：太大容易"矫枉过正"来回震荡，太小则学得太慢。本次训练采用**线性热身 + 线性衰减**策略：

- **第 1~3 轮（warmup 热身）**：学习率从 0.00047 逐步爬升到峰值 0.00133。刚开始训练时模型"心里没底"，先小步试探，避免一开始步子太大把预训练权重里已有的"看图能力"冲坏；
- **第 4~30 轮（线性衰减）**：学习率从峰值匀速降到 6.1×10^{-5}，越到后期步子越小，让模型在接近最优解时"精调"而不是来回跳动。

4.1.2 损失曲线：模型"错题本"的厚度

损失（loss）衡量"模型预测得有多离谱"，训练的目标就是让它不断变小：

- **cls_loss（分类损失）**：训练集从 5.87 **单调降至 0.84**，验证集同步从 9.20 降至 1.30。这是最漂亮的一条曲线——说明模型越来越会区分"摔倒/站立/坐着"，且**验证集**

与训练集同步下降，没有出现过拟合（过拟合的典型特征是训练集损失降、验证集损失反而升，即"只会背题、不会做题"）；

- box_loss（**定位损失**）：走势是"先升后降"——从 0.97 升到第 6 轮的 1.28，再稳步降到 0.72。前几轮的上升是 warmup 期间的正常现象：此时学习率正在爬坡、mosaic 数据增强又不断制造难样本，模型短暂"迷路"属正常，第 8 轮后便回归下降通道；
- **训练尾声仍在缓慢下降**，说明 30 轮尚未完全收敛，继续训练（50~100 epochs）还有提升空间（受 CPU 算力所限本次未做）。

4.1.3 精度曲线：从"见什么都报警"到精准识别

- 第 1 轮出现了一组很生动的数字：召回率（recall）= 1.0，精确率（precision）= 0.0035。意思是模型把画面中**所有东西都报成了目标**——宁可错杀一千，这种"乱报"状态在第 2 轮就迅速消失，精确率跳到 0.65；
- mAP@50（综合衡量检出精度的核心指标）从第 1 轮的 0.58 起步，中期在 0.68~0.81 之间震荡爬升，最终稳定在 **0.79**；更严格的 mAP@50-95 从 0.43 稳步升至 0.59，后 10 轮几乎单调上升。

4.2 自训练模型最终指标

30 轮训练后在 111 张验证集上的结果：

类别	Precision	Recall	mAP@50	mAP@50-95
fall（关键类）	**0.883**	**0.958**	**0.970**	0.762
standing	0.717	0.551	0.664	0.439
sitting	0.692	0.632	0.736	0.554
全部	0.764	0.714	0.790	0.585

对本系统而言，fall 是唯一需要高可靠的类别（漏报代价远大于误报），其 0.958 的召回率意味着验证集中 95.8% 的跌倒目标被成功检出，mAP@50 达 0.97，完全满足报警场景需求；standing/sitting 指标相对偏低，但这两类仅作为"干扰项"存在，不影响报警逻辑。

4.3 模型对比实验

4.3.1 微调前 vs 微调后：同一环境下的消融对比

最直接的证据来自"微调前后"的对照：同一段真实摔倒视频（3:24~3:45 片段，共 504 帧）、同一置信度阈值（0.25）、同一台机器，分别用微调前的官方预训练权重 `yolo26n.pt` 和微调后的 `best.pt` 跑一遍：

模型	输出类别	person 检出帧数	fall 检出帧数	fall 最高置信度
微调前 `yolo26n.pt`（COCO 80 类）	person / bicycle / …（无 fall 类）	504 / 504	0	—
微调后 `best.pt`（本项目）	fall / standing / sitting	—	244 / 504	0.94

结果很有说服力：微调前的模型**每一帧都稳稳地检测到了人**（504/504），说明主干网络的检测能力本来就在线——但它只能回答"这里有个人"，永远不可能回答"这个人摔倒了"，因为它的类别表里根本没有 fall。微调没有改变模型的"视力"，只是教会了它"摔倒长什么样"。这 244 帧的 fall 检出（未检出帧主要是站立行走和倒地后被遮挡的画面），就是 48 分钟微调训练换来的全部价值。

4.3.2 与开源跌倒模型的对比

在同一段摔倒视频上，与 GitHub 开源的 YOLOv11-Le2i 模型（同样基于公开跌倒数据训练的社区方案）对比：

模型	检出帧数	最高置信度	连续性
本项目 YOLO26n 微调	**45 帧**	**0.94**	摔倒期间连续命中
YOLOv11-Le2i（对照）	28 帧	0.68	断断续续

自训练模型在检出率与置信度上均明显优于对照模型。

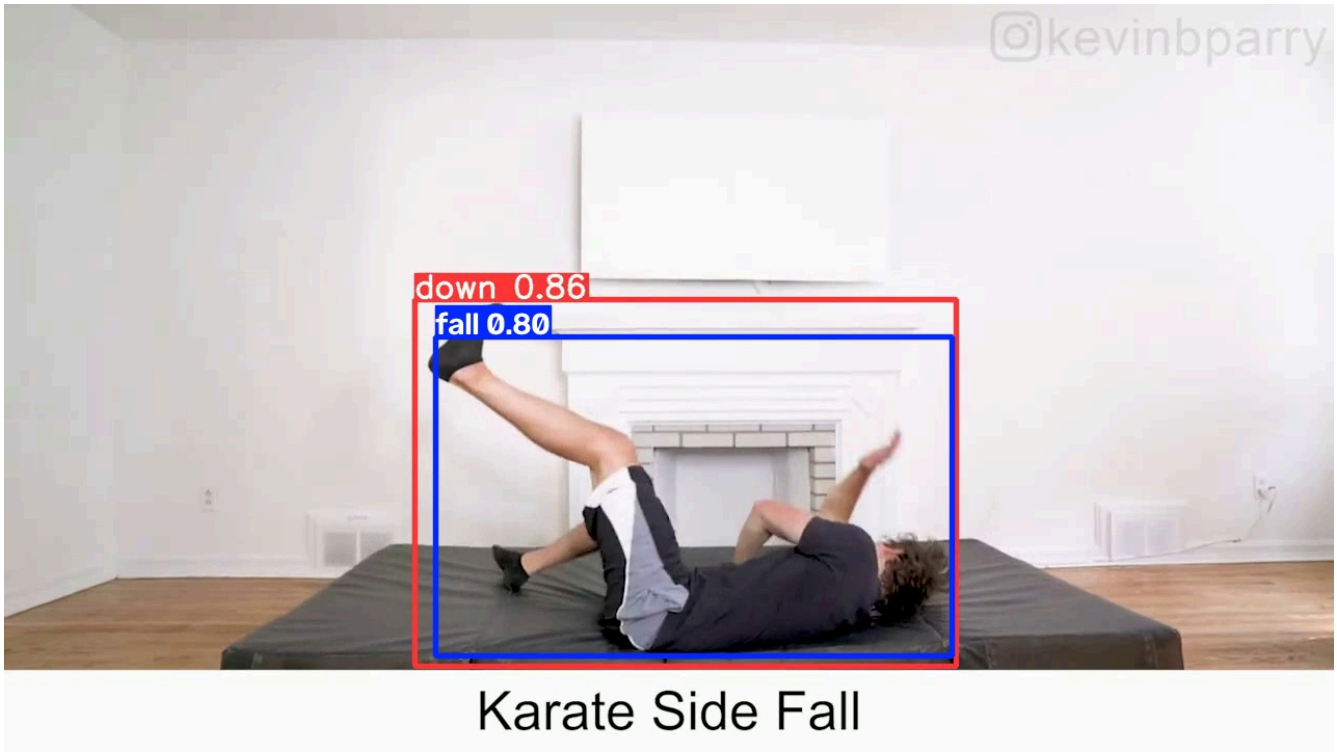


图 7　同一帧画面上两模型的实测输出：红框为对照模型（down，置信度 0.86），蓝框为本项目微调模型（fall，置信度 0.80），两者框选位置接近，但本项目模型在整段视频中的命中连续性更好

4.3.3 新模型档案（Model Card）

项目	内容
模型名称	`best.pt`（YOLO26n 跌倒检测微调模型）
架构	YOLO26n（nano 级最轻量规格）
权重体积	5.4 MB（对照 YOLOv11-Le2i 为 16 MB，仅为其 1/3）
输入尺寸	640 × 640
输出类别	fall（躺倒）/ standing（站立）/ sitting（坐着）
训练方式	COCO 预训练权重 + 374 张跌倒图片微调 30 epochs，纯 CPU 48 分钟
推理速度	CPU 约 67 ms/帧，满足准实时
适用场景	居家固定摄像头、平视/斜俯视角的跌倒检测

轻量是这个模型的实际优势：5.4 MB 的权重意味着它未来可以部署到树莓派、家用路由器这类边缘设备上，不依赖云端。

4.4 运动学确认实验

对同一段视频（3:10~3:52，包含多个真摔倒与缓慢躺下动作）的运动学判断日志：

时间	判定	峰速 v	角速度 ω	说明
03:13	✅ 摔倒	0.32	107°/s	双指标超标
03:20	🟣 缓慢躺下	0.08	36°/s	正确排除
03:24	✅ 摔倒	0.00	150°/s	前扑悬空，纯角速度捕获
03:26	✅ 摔倒	0.44	287°/s	侧摔
03:26	🟣 缓慢躺下	0.00	0°/s	正确排除
03:37~38	✅ 摔倒	0.47~0.74	221~240°/s	最剧烈的一次
03:43	🟣 缓慢躺下	0.00	0°/s	正确排除

03:24 的案例验证了双判据"或"逻辑的必要性：前扑悬空瞬间重心不再下降（v=0），但躯干高速旋转（ω=150°/s），单一下落速度判据会漏检。

4.5 2D vs 3D 姿势引擎对比实验（UR Fall 公开数据集）

为量化 2D 与 3D 姿势引擎的差异，在 UR Fall Detection Dataset（30 段真实摔倒 + 40 段日常活动，CC BY-NC-SA 学术许可）中选取 12 段（8 段摔倒含 2 段天花板俯拍机位 + 4 段日常活动），用**完全相同的判定链**（姿态持续 0.5 秒 → 躯干三点贴地 → 运动学确认），仅替换姿势引擎做对照：

视频	类型	2D 报警	3D 报警	3D 最小竖直性
fall-01-cam0	摔倒(出画)	未报	**3.8s**	0.02
fall-01-cam1	摔倒(俯拍)	未报	**0.5s**	0.00
fall-02-cam0	摔倒	未报	**2.9s**	0.03

视频	类型	2D 报警	3D 报警	3D 最小竖直性
fall-03-cam0	摔倒(朝镜头)	6.5s	未报	0.32
fall-03-cam1	摔倒(俯拍)	3.2s	0.5s	0.00
fall-05-cam0	摔倒	未报	4.9s	0.10
fall-08-cam0	摔倒	2.0s	1.6s	0.00
fall-12-cam0	摔倒	1.8s	2.3s	0.00
adl-01/02/05/10	日常活动	均未报	均未报	最低 0.16

指标	2D（YOLO26n-pose）	3D（MediaPipe）
摔倒检出率	4/8	7/8
日常活动误报	0/4	0/4

三点发现：

1. **3D 竖直性在 2D 盲区场景优势明显**：倒地后出画/遮挡及俯拍机位的 4 段视频 2D 全部漏检，3D 全部检出且最小竖直性 ≤ 0.10，分离度极好；
2. **3D 也存在中间态盲区**：fall-03-cam0（朝镜头摔）2D 检出而 3D 漏检——竖直性最低 0.32，处于 0.40 阈值边缘且持续不足 0.5 秒，"半倒半撑"姿态下单一阈值同样不可靠；
3. **多判据融合是零误报的关键**：日常视频中 3D 竖直性最低达 0.16（adl-10 为缓慢躺下），若仅用"竖直性 < 0.4"单判据必然误报；正是"躯干三点贴地 + 运动学速率"的组合把 4 段日常视频全部正确排除。值得注意的是 **2D 与 3D 漏检的样本互不重叠**——双引擎融合（任一引擎报警 + 贴地确认）理论上可达 8/8 检出且零误报，是后续工程方向。

4.6 已知不足

- 角速度对关键点抖动敏感：相邻帧仅差 0.04 秒，关键点抖动 5° 会产生 ~125°/s 的虚假角速度，需 3~5 帧滑动平均平滑；
- 多人拥挤 + 频繁镜头切换的影视片段中，追踪 ID 稳定性仍是瓶颈；

- 检测模型对训练数据未覆盖的摄像头角度（如正俯视）泛化能力待验证；
- 低分辨率（320×240）下关键点可用率显著下降（720p 约 94% vs 320×240 约 76%，倒地后 85% vs 41%），实际部署建议 720p 及以上；
- 3D 竖直性在"半倒半撑"过渡姿态存在中间态（0.3~0.5），阈值附近持续时长判定需要更细致的状态机或时序模型。

---

5. Vibe Coding 开发模式总结

本项目的代码几乎全部由 AI 助手编写与调试，人的角色集中于：提出需求（"要能区分睡觉"）、提供素材（测试视频）、验收效果（"摔倒时不识别"）、决策方向（"老师要求模型训练"）。

5.1 方法论经验

1. **现象描述质量决定调试效率**。"不识别"太模糊；"摔倒了但一直显示 OK"配合视频时间点，让 AI 一次定位到时间基准 bug。
2. **让 AI 写诊断工具，而不是盲猜**。逐帧测量日志比任何口头分析都快。
3. **里程碑式验收**：每轮迭代都用真实素材验收，问题暴露在早期。
4. **AI 也会引入 bug**（如编码损坏文件、时间基准混用），人必须保留最终验收权。

5.2 我的真实感受

说实话，一开始我对"AI 写代码"是半信半疑的，甚至有点觉得像"作弊"。但完整做完这个项目后，我的看法变了：**需求分析、方案取舍、效果验收、bug 现象的定位，这些真正难的部分其实还是人在做**；AI 把我从语法细节和查文档里解放出来，让我能把时间花在"想清楚到底要什么"上。

印象最深的是那个"时间基准 bug"——视频里人明明摔倒了系统却不报警，我盯着代码看根本看不出问题。最后是 AI 写了个诊断脚本，把每一帧的下坠速度、宽高比全部打印出来，数据一摆出来真相立刻大白。那一刻我真正理解了"**不要猜，要测量**"这句话。

也有踩坑的时候。有一次 AI 用命令行批量替换文本，直接把整个脚本搞成了乱码，之前的工作差点丢掉。从那以后我养成的习惯是：**每一轮改动都要立刻能跑、能验收**，重要版本自己留一份备份——不能完全信任 AI 的每一步操作。

另外一个真实的焦虑是：代码不是我逐行写的，给别人讲解时被问住过几次。后来我逼着自己把状态机和"几何+运动学双判据"的逻辑完整读懂了。我觉得"**能给别人讲清楚系统是怎么工作的**"，应该成为 Vibe Coding 的验收标准之一——否则做出来的东西不算真正属于自己。

如果用一个比喻总结这次课程项目：AI 是一个不知疲倦、知识渊博但偶尔会犯低级错误的搭档，而我是这个项目的"产品经理 + 测试 + 最终负责人"。这种分工，可能就是我们这代人以后工作的常态。

---

6. 总结与展望

本项目完成了一个可实际运行的老人跌倒检测系统，并产出三条互补的技术路线与一套自训练模型；在真实摄像头部署中迭代出"地板语义分割 + 三维姿势估计"的多判据融合方案，并在公开数据集上完成了 2D/3D 引擎的量化对比。后续方向：

- **双引擎融合**：2D 与 3D 姿势引擎漏检样本互不重叠，融合（任一报警 + 贴地确认）有望实现 8/8 检出且零误报；
- **模型迭代**：扩充数据（尤其自家监控视角的误报样本，如弯腰/深蹲类）、增加训练轮次、尝试更大模型（yolo26s/m）、冻结主干二次微调；
- **时序模型**：将关键点序列输入 LSTM/Transformer 做动作级分类，替代手工规则（3D 竖直性的"中间态"问题也适合用序列建模解决）；
- **工程落地**：远程报警链路（推送 + 自研 App + 监控画面）已打通；待完成阿里云语音电话开通（需企业资质）或 4G 电话模块接入（代码已就绪）、床区域标定上线、AMD 显卡通过 WSL2+ROCm 参与训练加速；
- **跌倒风险评估**：从"摔倒后报警"扩展到基于步态分析的长期风险预测。

参考文献

[1] Ultralytics YOLO 官方文档. https://docs.ultralytics.com/ [2] Charfi I, et al. Le2i Fall Detection Dataset. [3] Redmon J, et al. You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016. [4] HuggingFace 数据集: DeZan/fall-detection (MIT License). [5] GitHub: SyedBurhanAhmed/Real-Time-Fall-Detection-using-YOLO (对照模型, MIT License). [6] Kwolek B, Kepski M. Human fall detection on embedded platform using depth maps and wireless accelerometer. Computer Methods and Programs in Biomedicine, 117(3):489-501, 2014.（UR Fall Detection Dataset） [7] Xie E, et al. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. NeurIPS 2021. [8] Bazarevsky V, et al. BlazePose: On-device Real-time Body Pose tracking. CVPRW 2020.（MediaPipe 姿势模型）

附录：项目文件清单

文件	说明
`fall_detect.py`	姿势+规则版实时检测（含床区域标定、区域倒地计时）
`pose_action_demo.py`	**最终整合版主程序**（姿势识别+运动学确认+地板贴地校验+报警推送+监控视频流，默认摄像头模式）
`pose3d_demo.py`	3D 版演示（MediaPipe 33 三维关键点、竖直性判据、报警推送、视频流）
`alert_push.py`	手机推送模块（ntfy 管道，文字+现场截图）
`video_server.py`	MJPEG 监控视频流服务器（App/浏览器实时观看）
`call_module.py`	4G 电话模块拨号 + 中文 TTS 语音播报（预留硬件通道）
`aliyun_voice.py`	阿里云语音服务 TTS 电话报警（预留云通道）

文件	说明
`fall_camera.py`	自训练检测模型版报警程序（best.pt 直检 fall 类）
`fall_alert_app/`	**自研 Android App 完整源码**（Kotlin：报警接收、全屏警报、自动拨号、实时监控画面）
`dist/FallCamera/`	**最终版 Windows 可执行程序**（PyInstaller 打包，含模型与运行时，免 Python 环境）
`batch_compare_2d3d.py`	2D/3D 姿势引擎批量对比实验脚本
`compare_report.md`	对比实验数据报告
`pose_landmarker_full.task`	MediaPipe 姿势模型权重
`test_videos/`	UR Fall 数据集测试视频（含 rgb/ 裁剪版）
`train_fall.py`	模型微调训练脚本
`demo_model.py`	自训练模型视频演示脚本
`debug_analyze.py`	逐帧诊断工具（调试期编写）
`fall.yaml`	数据集配置
`fall_dataset/`	跌倒检测数据集（374 训练 / 111 验证）
`backups/`	模型与脚本备份
`runs/detect/runs_fall/train_yolo26n/weights/best.pt`	**自训练模型权重**
`runs/detect/runs_fall/train_yolo26n/results.png`	训练曲线
`runs/detect/runs_fall/val_demo/`	111 张验证集识别效果图
`fall_confirm_*.jpg`	运动学确认截图
`fig_pipeline.png`	图 1：系统整体流程图
`fig_yolo_arch.png`	图 2：YOLO26n 网络结构图
`fig_pose_head.png`	图 3：姿势估计（关键点识别）原理图

文件	说明
`fig_finetune.png`	图 4：迁移学习（微调）原理图
`fig_keypoints.png`	图 5：COCO 17 关键点示意图
`my_model_demo.jpg`	图 7：模型实测对比画面